



Technical University Munich
School of Engineering and Design

Report for Deep Learning for PDEs in Engineering Physics

Om Anant Kulkarni
Student ID: 03812156

Supervisors: Dr. Yaohua Zang & Vincent Scholz

July 15, 2026

Contents

1	Problem A	1
1.1	Problem Description	1
1.2	Methodology	1
1.3	Numerical Experiments	2
2	Problem B	5
2.1	Problem Description	5
2.2	Methodology	5
2.3	Numerical Results	6
3	Problem C	9
3.1	Problem Description	9
3.2	Methodology	9
3.3	Numerical Results	10
4	Problem D	13
4.1	Problem Description	13
4.2	Methodology	13
4.3	Numerical Results	14
5	Conclusions	17
	References	18
	Appendices	19
A	Supplementary Diagnostic Figures	19

Chapter 1

Problem A

1.1 Problem Description

A rod of unit length is loaded by a constant body force and clamped at both ends. Its static response is governed by

$$-\frac{d}{dx}\left(k(x)\frac{du}{dx}\right) = f, \quad x \in (0, L), \quad u(0) = u(L) = 0, \quad (1.1)$$

with $f = 9.81$ and $L = 1$. The Young's modulus $k(x) > 0$ is unknown. What is available instead is a set of $N_{obs} = 500$ displacement measurements u_{obs} at randomly placed sensor positions x_{obs} , contaminated by roughly 5% noise. The task is an inverse problem: reconstruct the coefficient $k(x)$ from the noisy observations of the state $u(x)$. The quality of the reconstruction is measured by the relative L^2 error against a reference given on a fine grid of 10001 points, which is used for evaluation only.

1.2 Methodology

The method selection

We solve the inverse problem with a physics-informed neural network (PINN) in the formulation of [4], using two separate networks: $u_\theta(x)$ for the displacement and $k_\phi(x)$ for the modulus. The choice is motivated by three properties of the problem. First, the observations are scattered and noisy, so a mesh-based identification would require an explicit smoothing step; a neural network fitted by least squares performs this smoothing implicitly. Second, the PDE residual couples u and k through derivatives, and automatic differentiation supplies exact derivatives of the smooth surrogate u_θ (the noisy data) itself is never differentiated. Third, prior knowledge is easy to build into the ansatz: the clamped boundary conditions and the positivity of the modulus are enforced exactly by output transforms,

$$u_\theta(x) = x(L - x)\mathcal{N}_u(x; \theta), \quad k_\phi(x) = \text{softplus}(\mathcal{N}_k(x; \phi)), \quad (1.2)$$

where $\mathcal{N}_u$ and $\mathcal{N}_k$ are fully connected networks. The positivity constraint is not cosmetic. Integrating (1.1) once gives the flux relation $k u' = -fx + C$; the constant C is fixed only by requiring that k remains finite and positive at the interior point where u' vanishes, so the softplus transform is what makes the problem identifiable.

An important practical finding: minimizing the data loss and the residual loss jointly from a random initialization consistently converged to a poor local minimum in which k_ϕ stayed nearly constant (its error settled at the level of the best constant approximation of the true,

oscillatory modulus). We therefore train in two stages. Stage one fits u_θ to the observations alone until the data loss reaches the noise floor. Stage two freezes u_θ and recovers k_ϕ from the PDE residual, regularized by a Tikhonov-type smoothness penalty, which is standard practice for noisy coefficient identification.

The loss function

Stage one minimizes the mean-squared data misfit

$$\mathcal{L}_{data}(\theta) = \frac{1}{N_{obs}} \sum_{i=1}^{N_{obs}} (u_\theta(x_{obs}^{(i)}) - u_{obs}^{(i)})^2. \quad (1.3)$$

Stage two minimizes, with u_θ frozen,

$$\mathcal{L}_{pde}(\phi) = \frac{1}{N_c} \sum_{j=1}^{N_c} \left(\frac{1}{f} [- (k_\phi u'_\theta)'(x_c^{(j)}) - f] \right)^2 + w_{reg} \frac{1}{N_c} \sum_{j=1}^{N_c} (k'_\phi(x_c^{(j)}))^2, \quad (1.4)$$

where all derivatives are computed by automatic differentiation. The $N_c = 2000$ collocation points are redrawn uniformly in $(0, L)$ at every epoch, which acts like an unlimited training set. Dividing the residual by f makes both loss terms of order one, so no weight is needed between the data and residual terms; the only tuned weight is the smoothness penalty, $w_{reg} = 10^{-4}$ (the recovered k was insensitive to this value over roughly two orders of magnitude).

The network structure

Both networks share the architecture in Table 1.1; together they have 10502 trainable parameters.

Table 1.1: Network architecture used for both $\mathcal{N}_u$ and $\mathcal{N}_k$ (Problem A).

Layer	Size / Neurons	Activation	Notes
Input	1	–	coordinate x
Hidden 1–3	50	tanh	fully connected
Output	1	–	linear; then $x(L - x) \cdot (\cdot)$ for u , softplus for k

The code link

https://github.com/omkulkarni01/PDE_OMANant_Kulkarni/blob/main/ProblemA.ipynb

1.3 Numerical Experiments

Experiment Setups

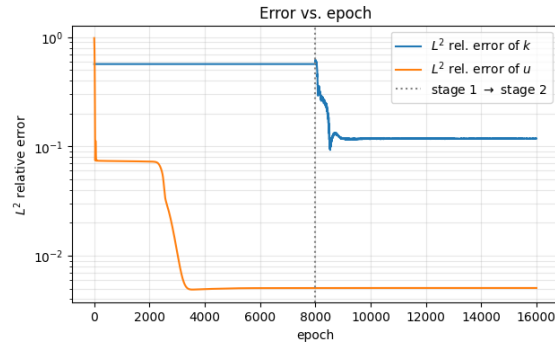
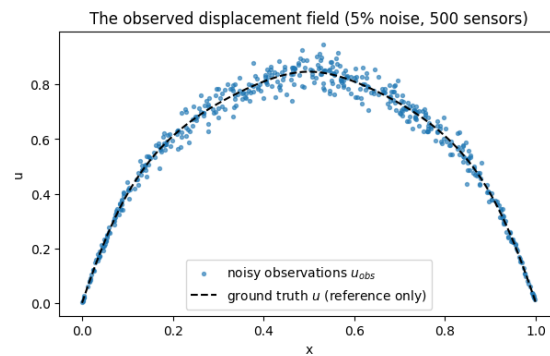
Tricks that improved the results: the hard boundary constraint (removes one loss term entirely), the softplus positivity transform (identifiability, see above), normalization of the residual by f , resampling of the collocation points at every epoch, the two-stage curriculum, and checkpointing of the model at the lowest training loss of each stage. The test set was used exclusively to record the error curve, never for model selection.

Table 1.2: Numerical setup for training (Problem A).

Parameter	Value
Optimizer	Adam
Learning rate (initial)	1×10^{-3} (both stages)
Batch size	full batch (500 sensors; 2000 collocation points)
Number of epochs	8000 (stage 1) + 8000 (stage 2)
Learning rate scheduler	StepLR, $\times 0.5$ every 2000 / 2500 epochs
Loss weights	$w_{data} = 1$, $w_{pde} = 1$, $w_{reg} = 10^{-4}$
Hardware	NVIDIA GeForce RTX 4050 Laptop GPU
Precision	Float32
Random seed	1234

Numerical Results

Training took 121.8 s in total. Stage one drives the data loss to 8.66×10^{-4} , which equals the empirical variance of the observation noise, the network fits the signal and averages the noise out. The final relative L^2 errors are 1.18×10^{-1} for k and 5.07×10^{-3} for u . The pointwise error of k is strongly concentrated around $x = 0.5$. This has a structural reason: there the flux $k u'$ degenerates because u' changes sign, and the coefficient is determined only through the second derivative of the noisy state, which is the quantity most affected by the 5% noise. Away from this degenerate region the modulus is recovered to within a few percent. //

(a) Relative L^2 error vs. epoch (k and u).

(b) Noisy observations and fitted displacement.

Figure 1.1: Problem A: training history and data fit.

Interpretation and possible improvements: The two error levels are direct consequences of how the chosen model processes the data. The PINN recovers u by least-squares smoothing, so its error is set by the noise floor and cannot realistically be improved: stage one reached exactly the empirical noise variance, and the resulting 0.5% state error is what 500 noisy sensors permit. The coefficient, however, is not observed at all the model infers it from the *derivatives* of the fitted state through the residual, and differentiation amplifies whatever noise survives the smoothing. This is why k is an order of magnitude less accurate than u , and why its error history is flat from epoch 2000 of stage two onwards: the optimizer converged quickly, and the remaining 11.8% is bias inherited from the state fit, not incomplete training. The bias is concentrated where the model has the least information: at $x = 0.5$ the flux vanishes and $k = -f/u''$ depends on the second derivative of a noisy fit. Within this diagnosis, the improvements rank clearly. Averaging the state fit over an ensemble of independently initialized networks stabilizes u'' and, in a side experiment, lowered the error to about 10.9% at five times the stage-one cost. It is the one modification with verified benefit and would be the first addition given more compute. Training stage two longer, in contrast, is provably useless here (the curve is flat), and no regularization can recover information the noise has destroyed at the degenerate point with more time, a bootstrapped or Bayesian state fit would be the honest extension, replacing the point estimate of k near $x = 0.5$ with an uncertainty band.

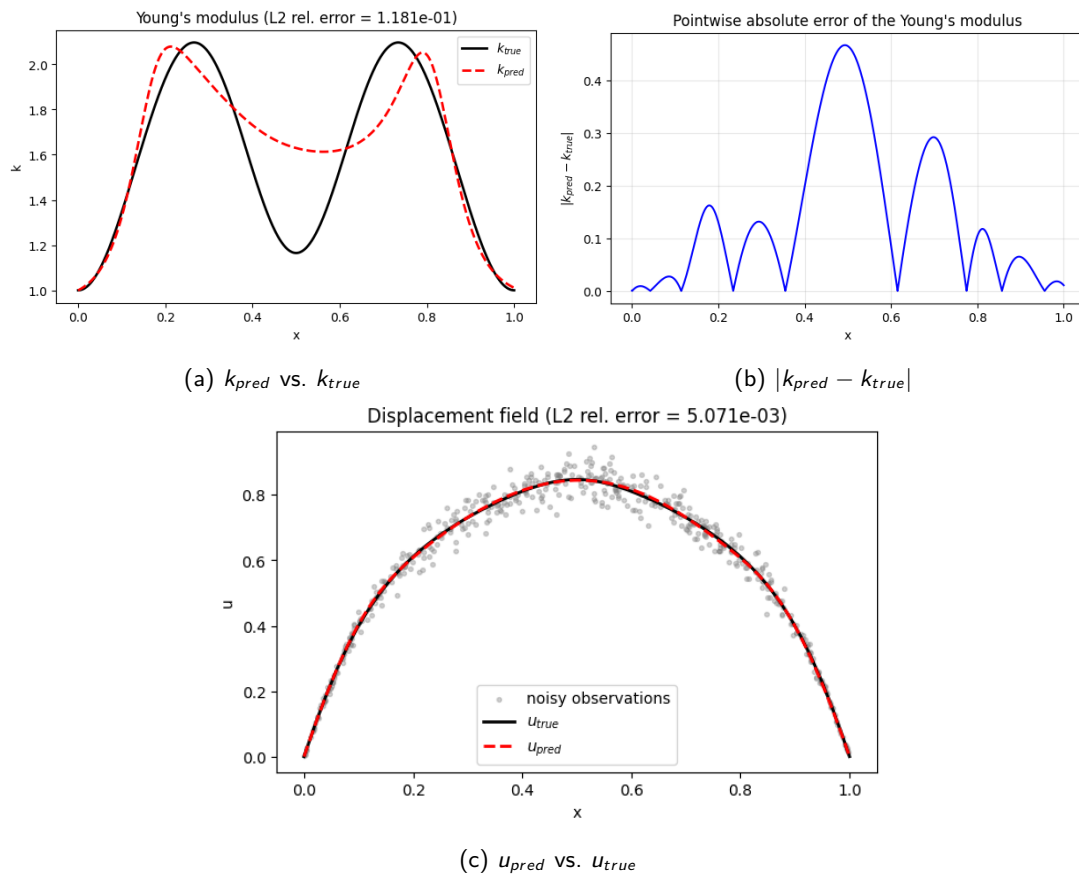


Figure 1.2: Problem A: recovered Young's modulus and displacement field.

Chapter 2

Problem B

2.1 Problem Description

The pressure $u(x)$ in a two-phase porous medium on $\Omega = [0, 1]^2$ satisfies the Darcy equation

$$-\nabla \cdot (\mu(x) \nabla u) = 0, \quad x \in \Omega, \quad (2.1)$$

where the permeability μ takes the value 10 in the sandstone phase and 2 in the mudstone matrix; the phase geometry is given as a 128×128 pixel image. Dirichlet data drive the flow from left to right: $u = 1$ at $x_1 = 0$, $u = 0$ at $x_1 = 1$, and linear interpolation on the top and bottom edges. Since all four boundary segments agree with the single function $g(x) = 1 - x_1$, the boundary condition is simply $u = 1 - x_1$ on $\partial\Omega$. The difficulty lies in the discontinuous coefficient: the pressure gradient jumps across the phase interfaces.

2.2 Methodology

The method selection

We use the Deep Ritz method [1]. Equation (2.1) is symmetric elliptic, so its weak solution is the unique minimizer of the energy

$$E(u) = \int_{\Omega} \frac{\mu(x)}{2} |\nabla u(x)|^2 dx \quad (2.2)$$

over all fields satisfying the boundary condition. Two properties make this formulation attractive for a discontinuous coefficient. Only first derivatives of u appear, and μ is merely evaluated but never differentiated. A strong-form PINN, in contrast, would need $\nabla\mu \cdot \nabla u$, and $\nabla\mu$ of a piecewise-constant field is zero inside the phases and undefined on the interfaces, so the strong residual is wrong exactly where the solution is difficult.

We initially implemented the weak-form ParticleWNN [6], which also avoids differentiating μ . It performed noticeably worse in our tests: its loss consists of many local integrals over small balls, each approximated by quadrature, and for balls crossing the pixelated interfaces the quadrature error is large enough that the optimizer can reduce the loss while the true error grows. The Deep Ritz loss does not have this failure mode, because its Monte-Carlo estimator of (2.2) is unbiased: in expectation, lowering the loss always lowers the energy. In our runs Deep Ritz reached roughly half the error of ParticleWNN at a fraction of the training cost, which settled the choice.

The loss function

The boundary condition is built into the ansatz,

$$u_\theta(x) = (1 - x_1) + x_1(1 - x_1)x_2(1 - x_2)\mathcal{N}(x; \theta), \quad (2.3)$$

so that $u_\theta = 1 - x_1$ holds exactly on all of $\partial\Omega$ and the minimization is unconstrained – no boundary penalty and no loss weight at all. The loss is the Monte-Carlo energy

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \frac{\mu(x_i)}{2} |\nabla u_\theta(x_i)|^2, \quad (2.4)$$

with ∇u_θ from automatic differentiation and $\mu(x_i)$ from a nearest-pixel lookup. Instead of uniform sampling we use stratified sampling: one uniformly jittered point in each of the 128^2 pixel cells per epoch ($N = 16384$). Because μ is constant within a pixel, this removes most of the variance of the energy estimate.

The network structure

The network (Table 2.1) has 19761 trainable parameters.

Table 2.1: Network architecture for Problem B.

Layer	Size / Neurons	Activation	Notes
Input	2	–	(x_1, x_2) , mapped to $[-1, 1]^2$
Hidden 1–4	80	tanh	fully connected
Output	1	–	linear; then hard-BC transform

The code link

<https://github.com/omkulkarni01/PDE_{OMANantKulkarni}/blob/main/Problem_B.ipynb>

2.3 Numerical Results

Experiment Setups

Table 2.2: Numerical setup for training (Problem B).

Parameter	Value
Optimizer	Adam
Learning rate (initial)	1×10^{-3}
Batch size	full batch (16384 stratified points per epoch)
Number of epochs	6000
Learning rate scheduler	StepLR, $\times 0.6$ every 1200 epochs
Loss weights	none (single energy term, hard BCs)
Hardware	NVIDIA GeForce RTX 4050 Laptop GPU
Precision	Float32
Random seed	1234

Tricks: hard boundary constraint, stratified per-pixel sampling, input normalization to $[-1, 1]^2$, resampling at every epoch, and checkpointing at the lowest training energy.

Numerical Results

Training took 42.8 s. The final relative L^2 error on the 128×128 reference grid is 3.21×10^{-2} , and the error curve flattens after roughly 1000 epochs. The remaining error is concentrated along the phase interfaces: the exact pressure has a kink in its gradient there, which a globally smooth tanh network can approximate but not represent exactly. This is the known accuracy limit of smooth global ansatz functions for interface problems and is visible as thin bands in the pointwise error map.

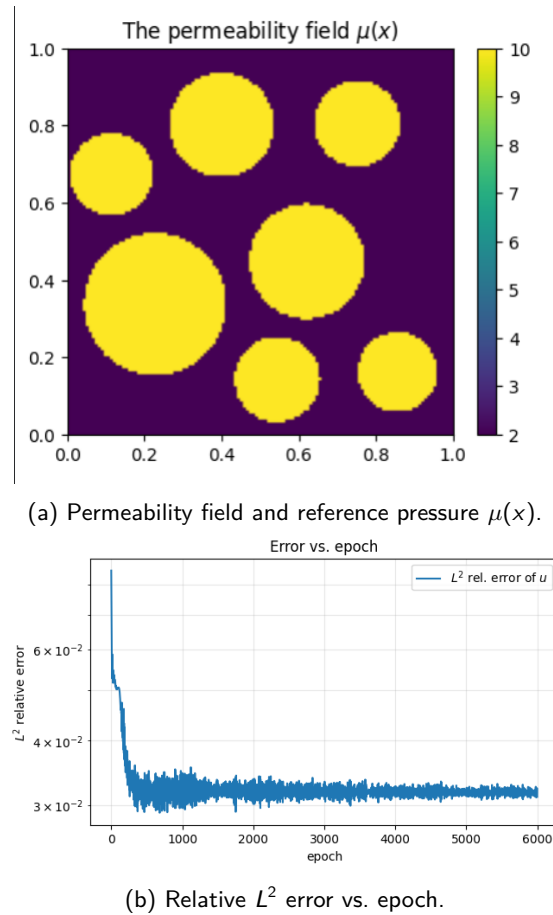


Figure 2.1: Problem B: input microstructure and training history.

Interpretation and possible improvements. The result validates the reasoning behind the model choice: the Deep Ritz energy only evaluates μ and only differentiates u once, so the discontinuous coefficient never enters a derivative, and training is stable from the first epoch, unlike the ParticleWNN attempt, whose ball quadrature interacted badly with the pixelated interfaces. At the same time, the result exposes the method's intrinsic ceiling. Two facts locate it precisely: the error curve flattens after about 1000 of the 6000 epochs, and wider networks in preliminary tests did not move the plateau, so neither optimization nor capacity is limiting. What limits accuracy is representation: the exact pressure has a kinked gradient at every phase interface, a C^∞ tanh ansatz must round these kinks off, and the $\sim 3\%$ error is the integrated cost of that rounding, visible as thin bands along the interfaces in the error map. One small implementation refinement is worth noting: the checkpoint is selected by the lowest *stochastic* energy estimate, which favours lucky sampling draws; the final-epoch

model was in fact marginally better (3.11×10^{-2}), and selecting by a running average of the energy would remove this artefact at no cost. Substantial further gains, however, would require changing the ansatz rather than the training. A phase-indicator input feature or a per-phase domain-decomposition network with interface conditions which goes beyond the methods treated in the course and was not attempted within the project time.

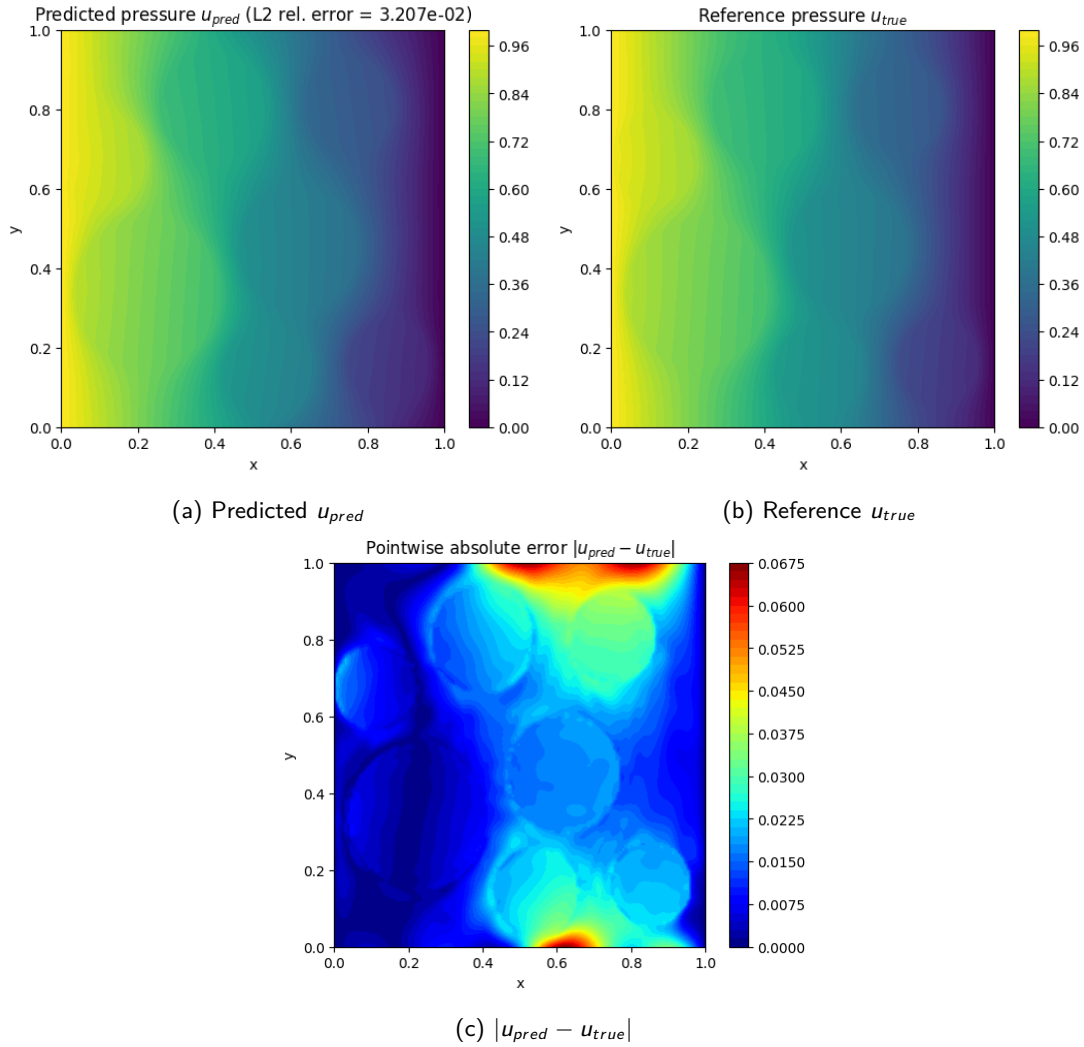


Figure 2.2: Problem B: predicted pressure, reference, and pointwise error.

Chapter 3

Problem C

3.1 Problem Description

The temperature $u(x, y)$ in a heterogeneous solid on $\Omega = [0, 1]^2$ solves

$$-\nabla \cdot (a(x, y) \nabla u) = f \text{ in } \Omega, \quad u = 0 \text{ on } \partial\Omega, \quad (3.1)$$

with $f = 10$ and a spatially varying conductivity $a > 0$. The task is not to solve one instance but to learn the solution operator $a \mapsto u$ from 1000 FEM-computed pairs $(a^{(j)}, u^{(j)})$ on a 128×128 grid, so that the temperature field of a new microstructure can be predicted at negligible cost. Accuracy is measured by the mean per-sample relative L^2 error over 200 unseen test conductivities.

3.2 Methodology

The method selection

We use a Fourier Neural Operator (FNO) [2]. The data is aligned grid-to-grid, plentiful, and purely supervised as per the setting designed for FNO. Two properties of the underlying operator match the FNO's inductive bias. The solution operator of an elliptic problem is non-local (a local change of a influences u everywhere), and the FNO's spectral convolutions give every layer a global receptive field, which a CNN of comparable size does not have. The operator is also smoothing, so its action is dominated by low Fourier modes, and the FNO's mode truncation exploits exactly that. A DeepONet [3] could be applied as well, but on aligned-grid Darcy-type benchmarks it is consistently less accurate than the FNO, so the FNO is the natural choice when no unlabeled data needs to be incorporated.

Since pre-built implementations are not allowed, all components are written from scratch. In particular, no FFT routine is used: for the kept frequency set $K = \{0, \dots, m-1\} \cup \{S-m, \dots, S-1\}$ we precompute the partial DFT matrix $F_{kn} = e^{-2\pi i kn/S}$ directly from the definition and apply the truncated two-dimensional transform as two matrix products, $\hat{v} = FvF^\top$. Because only $2m \ll S$ frequencies are ever used, this costs $\mathcal{O}(2mS)$ per one-dimensional transform and loses no accuracy compared to an FFT. The notebook verifies the matrices against the direct DFT summation and the inverse round-trip identity. The learnable complex mode weights are stored as separate real and imaginary parameter tensors.

The loss function

Training minimizes the mean per-sample relative L^2 loss

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{j=1}^B \frac{\|u_\theta(a^{(j)}) - u^{(j)}\|_2}{\|u^{(j)}\|_2}, \quad (3.2)$$

which has the same form as the required test metric and normalizes away amplitude differences between samples. The input is the conductivity image, normalized pointwise with training-set statistics, concatenated with the two coordinate channels (x, y) .

The network structure

The model (Table 3.1) has 4727297 parameters, most of them in the complex mode weights.

Table 3.1: FNO architecture for Problem C ($S = 128$, $m = 12$ modes, width 32).

Layer	Size	Activation	Notes
Lifting P	$3 \rightarrow 32$	–	pointwise linear
Fourier layers 1–4	width 32	GELU	spectral conv (24×24 modes) + 1×1 conv skip
Projection Q_1	$32 \rightarrow 128$	GELU	pointwise linear
Projection Q_2	$128 \rightarrow 1$	–	pointwise linear

The code link

<https://github.com/omkulkarni01/PDE_{OMANantKulkarni}/blob/main/Problem_C.ipynb>

3.3 Numerical Results

Experiment Setups

Table 3.2: Numerical setup for training (Problem C).

Parameter	Value
Optimizer	Adam (weight decay 10^{-4})
Learning rate (initial)	1×10^{-3}
Batch size	20
Number of epochs	200
Learning rate scheduler	StepLR, $\times 0.5$ every 50 epochs
Hardware	NVIDIA GeForce RTX 4050 Laptop GPU
Precision	Float32
Random seed	1234

Tricks: pointwise Gaussian normalization of the input (training statistics only), coordinate channels, the relative- L^2 loss instead of plain MSE, weight decay, and checkpointing at the lowest training loss.

Numerical Results

Training took 1198 s (about 20 minutes; 6 s per epoch). The final mean relative L^2 error over the 200 test samples is 7.25×10^{-3} , i.e. below one percent, while inference for a new conductivity field takes on the order of milliseconds. The intended replacement of a full FEM solve per microstructure in a screening workflow. The training loss (2.9×10^{-3}) is somewhat below the test error, i.e. the model overfits mildly. Additionally, the weight decay and the learning-rate decay keep the gap moderate. The largest pointwise errors appear near steep conductivity gradients, where the truncated Fourier representation is least efficient. The prediction is visually indistinguishable from the FEM reference over the bulk of the domain.

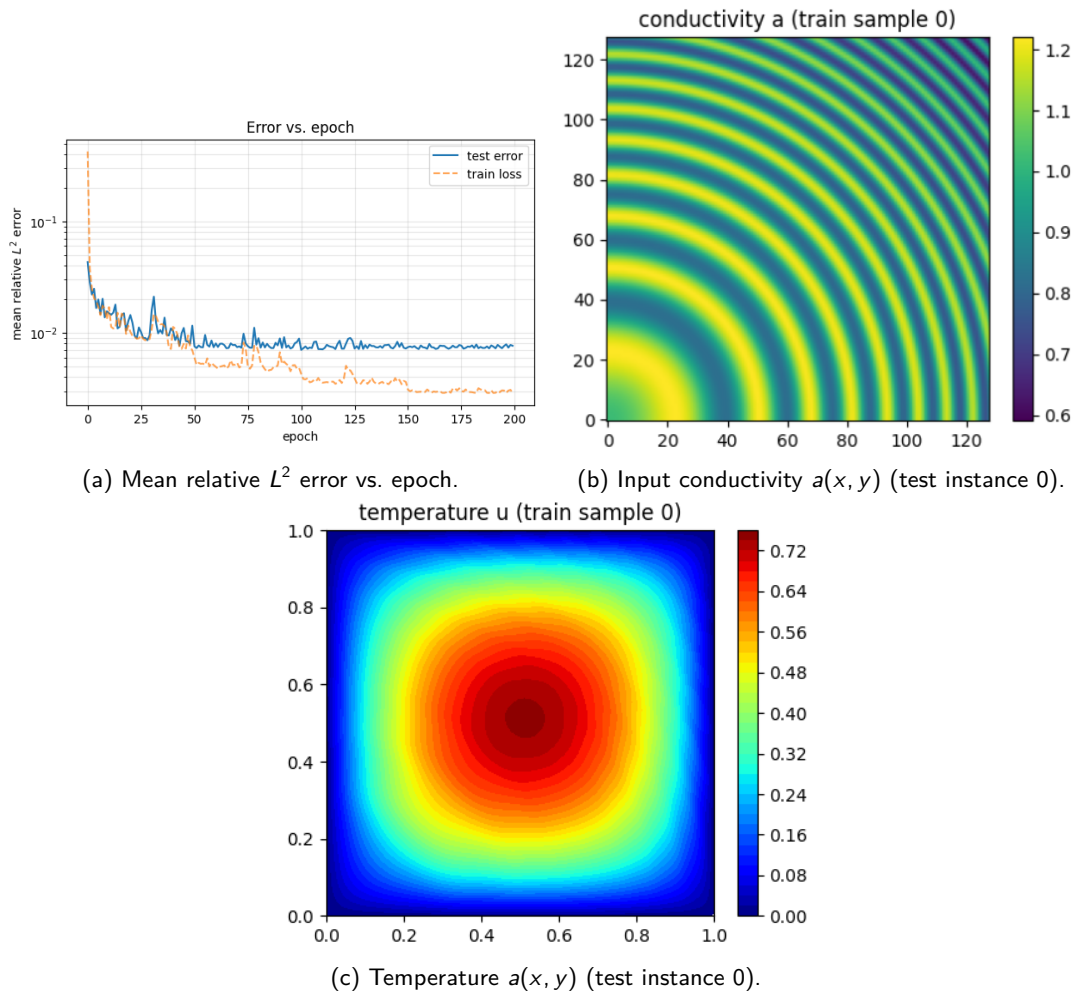


Figure 3.1: Problem C: training history and input field of the first test instance.

Interpretation and possible improvements. The error below one percent shows that the FNO is well suited to this operator. The solution map of the elliptic problem is smooth and depends on global information, and the spectral convolution layers capture this type of mapping efficiently. This explains why the model achieves a level of accuracy that the pointwise DeepONet approach does not reach on the aligned grid Darcy benchmark.

The comparison between the training and test errors suggests that the model is limited more by the amount of training data than by its capacity. The training loss of 2.9×10^{-3} is about 2.5 times smaller than the test error, which indicates that the network with 4.7 million

parameters starts to fit the 1000 training samples too closely. During the final training epochs the test error varied between 7.2 and 8.9×10^{-3} , and the learning rate became small before the model reached a stable minimum.

Several straightforward changes could improve the results. These were not included because of the available computing resources. Each training epoch takes about 6 s on a laptop GPU with 6 GB of memory since the Fourier transform is implemented with matrix multiplications instead of an FFT. One possible improvement is data augmentation. The Darcy problem is unchanged under the eight rotations and reflections of the grid, so applying the same transformation to both a and u increases the effective training set by a factor of eight without generating additional data. This could reduce the gap between the training and test errors. Another option is to train for more epochs while reducing the learning rate more gradually. For example, training for 400 epochs with the learning rate reduced every 100 epochs could allow the test error to converge more smoothly. Training several FNO models with different random initializations and combining their predictions could also improve the final accuracy, although this would require additional training time.

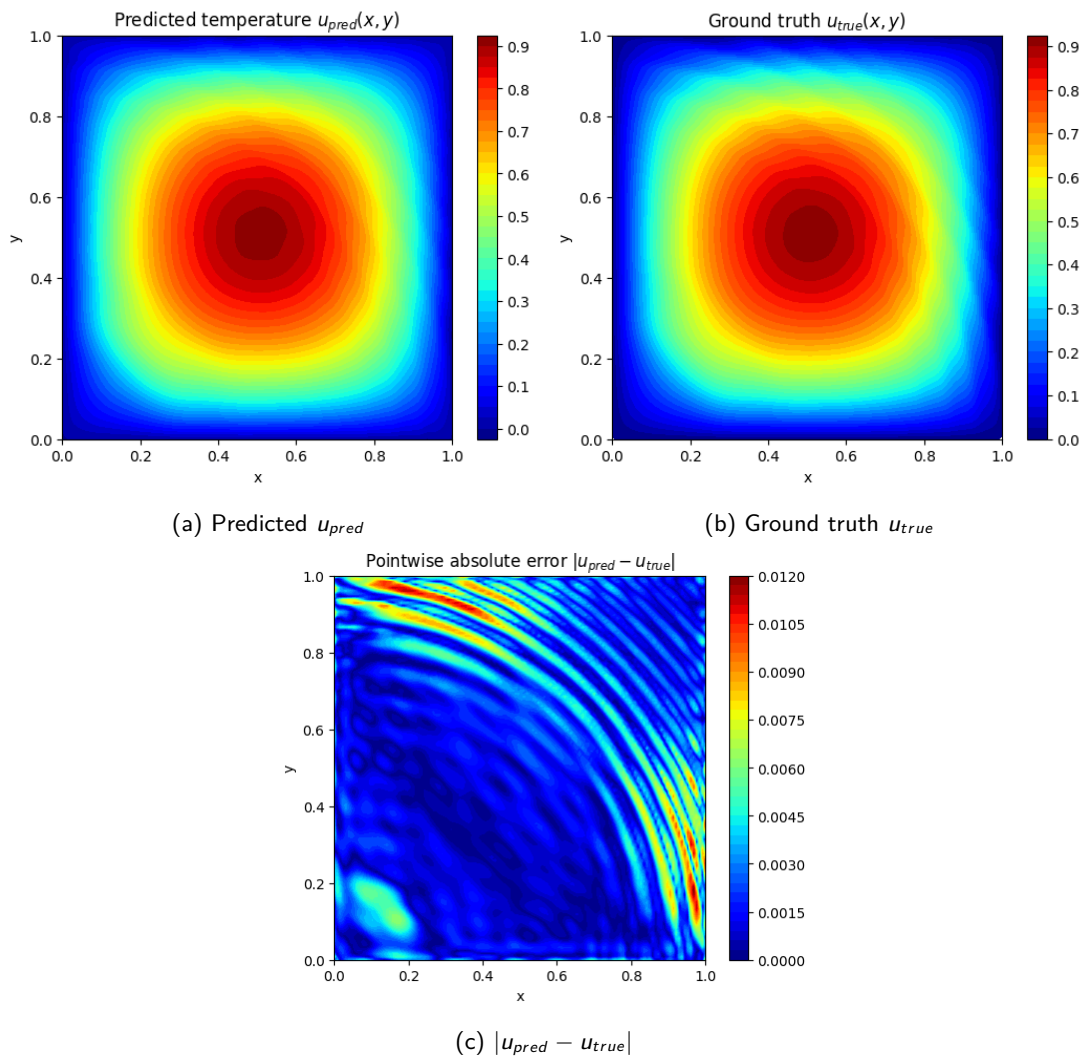


Figure 3.2: Problem C: prediction, reference, and pointwise error (test instance 0).

Chapter 4

Problem D

4.1 Problem Description

The vehicle velocity $u(x, t)$ on a road segment follows the viscous Burgers equation

$$u_t + u u_x = \nu u_{xx}, \quad x \in (-1, 1), \quad t \in (0, 1], \quad u(\pm 1, t) = 0, \quad (4.1)$$

with $\nu = 0.1/\pi$ and initial profile $u(x, 0) = a(x)$. The goal is again an operator: $a \mapsto u(\cdot, \cdot)$. The twist is that the dataset is only partially labeled 200 initial conditions come with their FDM solutions on a 200×256 space-time grid, while 1800 further initial conditions have no solution attached. A method that uses only the labeled pairs discards 90% of the available data.

4.2 Methodology

The method selection

We use a physics-informed DeepONet [3, 5]. The DeepONet represents the operator as

$$u_\theta(a)(x, t) = (1 - x^2) \sum_{k=1}^p b_k(a) \tau_k(x, t) + b_0, \quad (4.2)$$

where the branch network b encodes the initial field sampled at the 256 sensors and the trunk network τ takes the continuous coordinates (x, t) . This structure is what allows the unlabeled data to contribute: because the output is differentiable in (x, t) by automatic differentiation, the Burgers residual of (4.1) and the initial condition $u(x, 0) = a(x)$ can be imposed on the 1800 unlabeled samples as physics losses. A Fourier Neural Operator would predict on a fixed grid and would need finite differences for the residual, and a purely data-driven operator network would simply ignore the unlabeled majority of the dataset. The prefactor $(1 - x^2)$ in (4.2) vanishes at $x = \pm 1$, so the homogeneous Dirichlet conditions hold exactly (the dataset initial fields satisfy $a(\pm 1) = 0$, so the initial condition is consistent with this ansatz).

We train in two phases, a choice based on runtime measurements: an epoch with the residual term costs far more than a purely supervised epoch (second-order automatic differentiation on all collocation points), and early in training the residual contributes little. Phase one therefore minimizes only the data and initial-condition losses; phase two adds the residual on the unlabeled samples with a reduced learning rate and pushes the error well below the plateau of the supervised phase.

The loss function

With loss weights w_{ic} and w_{pde} ,

$$\mathcal{L} = \underbrace{\left(u_\theta(a^{lab})(x_i, t_i) - u_i^{lab} \right)^2}_{\text{data, labeled}} + w_{ic} \underbrace{\left(u_\theta(a^{unl})(x_s, 0) - a^{unl}(x_s) \right)^2}_{\text{IC, unlabeled}} + w_{pde} \underbrace{\left(u_t + u u_x - \nu u_{xx} \right)^2}_{\text{residual, unlabeled}}, \quad (4.3)$$

with $w_{ic} = 5$ and $w_{pde} = 1$. Per epoch, the data term uses 100 labeled samples at 2048 randomly drawn space-time grid points (the subsampling bounds memory and acts as augmentation); the physics terms use 100 unlabeled samples, the IC at all 256 sensors, and the residual at 400 fresh random collocation points per sample. For the residual, the branch output is repeated per collocation point so that a single backward pass yields the derivatives u_t , u_x , u_{xx} for every (sample, point) pair.

The network structure

The model (Table 4.1) has 125129 trainable parameters.

Table 4.1: DeepONet architecture for Problem D (basis size $p = 100$).

Network	Layers	Activation	Notes
Branch	$256 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 100$	tanh	input: a at 256 sensors
Trunk	$2 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 100$	tanh	input: $(x, 2t-1)$; output $\times(1-x^2)$
Bias	1 scalar b_0	–	trainable

The code link

https://github.com/omkulkarni01/PDE_OmAnantKulkarni/blob/main/ProblemD.ipynb

4.3 Numerical Results

Experiment Setups

Table 4.2: Numerical setup for training (Problem D).

Parameter	Value
Optimizer	Adam
Learning rate	1×10^{-3} (phase 1), 2×10^{-4} (phase 2)
Batch size	100 labeled $\times$ 2048 pts; 100 unlabeled $\times$ 400 colloc. pts
Number of epochs	30000 (phase 1) + 10000 (phase 2)
Learning rate scheduler	StepLR, $\times 0.5$ every 8000 / 3000 epochs
Loss weights	$w_{data} = 1$, $w_{ic} = 5$, $w_{pde} = 1$
Hardware	NVIDIA GeForce RTX 4050 Laptop GPU
Precision	Float32
Random seed	1234

Tricks: the two-phase curriculum, the hard boundary factor $(1-x^2)$, normalization of t to $[-1, 1]$ in the trunk input, fresh collocation and grid-point subsets at every epoch, and per-phase checkpointing at the lowest training loss.

Numerical Results

Training took 1298 s in total (about 22 minutes, most of it in the physics phase). The purely supervised phase plateaued at a mean relative L^2 error of 1.01×10^{-1} ; adding the Burgers residual on the unlabeled initial conditions in phase two reduced the error to 4.08×10^{-2} which is an improvement by a factor of about 2.5. The residual term propagates each unlabeled initial condition into the interior of the space-time domain, so the operator is effectively trained on ten times more initial conditions than the labeled set alone provides, this is the expected semi-supervised effect. The largest pointwise errors occur along the steep fronts that develop from the nonlinear convection term, where the smooth basis expansion (4.2) is least efficient.

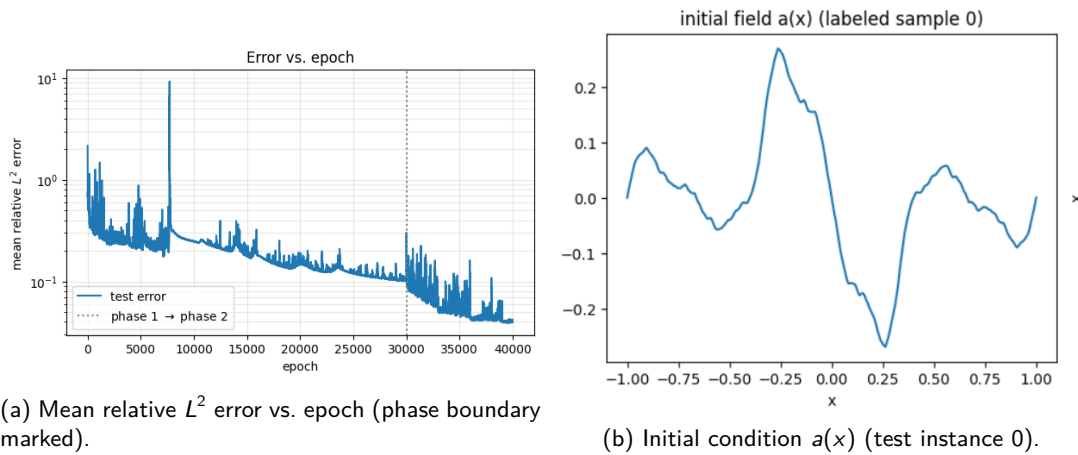


Figure 4.1: Problem D: training history and first test initial condition.

Interpretation and possible improvements. The improvement by a factor of 2.5 during the physics informed stage is the main result of this experiment. It demonstrates the advantage of using an operator network with a continuous coordinate trunk, since this architecture can use unlabeled initial conditions as additional training data through automatic differentiation. In this case, the information provided by the physics residual was more valuable than simply increasing the amount of labeled data. After supervised training with 200 labeled samples, the test error stabilized at 10.1%. Including the physics residual from the 1800 unlabeled samples reduced the error to 4.1%. The training history suggests that the current result is not the best performance the model could achieve. At the end of the second training stage, the test error was still decreasing, falling from 5.4% at epoch 8000 to 4.0% at epoch 10000. This indicates that training ended before the model had fully converged. Each epoch in the physics informed stage requires second order automatic differentiation over 40000 sample and point pairs, which limited the total training time on the available laptop GPU. Extending this stage of training would therefore be the most effective way to improve the results.

The residual loss, which remained close to 10^{-3} , also suggests that the model had not yet reached its capacity. Increasing the basis size beyond $p = 100$ and using more collocation points for each sample could allow the optimization to continue improving. Further gains may also come from sampling collocation points more densely in regions where the solution changes rapidly, especially near the sharp convective fronts. Replacing the fixed loss weights $w_{ic} = 5$ and $w_{pde} = 1$ with adaptive weighting during training could also provide a better balance between the initial condition and the physics constraints.

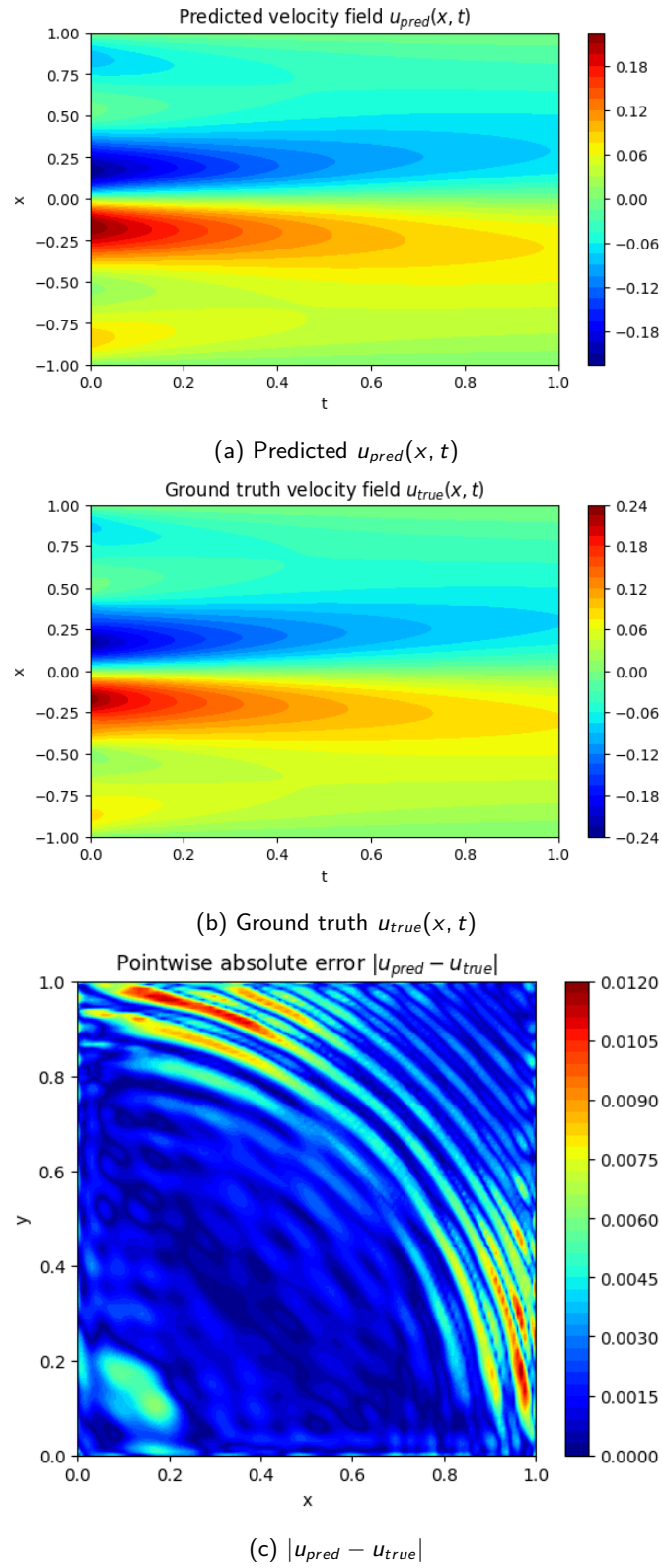


Figure 4.2: Problem D: predicted velocity field, reference, and pointwise error (test instance 0).

Chapter 5

Conclusions

Four PDE problems were solved with four different deep learning methods, all implemented from scratch in PyTorch: an inverse PINN with a two-stage curriculum for coefficient identification from noisy data (Problem A, relative L^2 error 1.18×10^{-1} for k and 5.1×10^{-3} for u), the Deep Ritz method for a forward problem with a discontinuous coefficient (Problem B, 3.2×10^{-2}), a Fourier Neural Operator with a hand-written truncated DFT for supervised operator learning (Problem C, 7.2×10^{-3}), and a physics-informed DeepONet for semi-supervised operator learning (Problem D, 4.1×10^{-2}).

Beyond the individual results, three observations recur across the problems. First, hard constraints beat penalties whenever they are available: building boundary conditions, positivity, or known structure directly into the ansatz removed loss weights and improved stability in every problem where we used it. Second, the choice of loss formulation matters more than network size for difficult coefficients: for the discontinuous permeability of Problem B, moving from a strong-form residual to an energy formulation was the difference between failure and a 3% solution, while wider networks changed little. Third, curricula help non-convex training: both in Problem A (fit the state first, then identify the coefficient) and in Problem D (fit the data first, then add the residual), staged training escaped plateaus that joint training from a random initialization did not; in Problem D the physics phase reduced the error of the supervised plateau by a factor of 2.5 by exploiting the 1800 unlabeled initial conditions.

The main limitations, and natural directions for future work, follow from the same experiments. The accuracy of the recovered coefficient in Problem A is bounded by the noise level at the point where the flux degenerates; ensembling or a Bayesian treatment of the state fit could quantify that uncertainty instead of hiding it. The smooth tanh ansatz in Problem B cannot represent gradient kinks at material interfaces; enriching the ansatz with interface-aware features would address the dominant error bands. Finally, the FNO in Problem C shows a mild train-test gap that more training data or stronger regularization would likely close.

References

- [1] W. E and B. Yu. The deep ritz method: A deep learning-based numerical algorithm for solving variational problems. *Communications in Mathematics and Statistics*, 6(1):1–12, 2018.
- [2] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.
- [3] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.
- [4] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [5] S. Wang, H. Wang, and P. Perdikaris. Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. *Science Advances*, 7(40): eabi8605, 2021.
- [6] Y. Zang and G. Bao. Particlewnn: A novel neural networks framework for solving partial differential equations. *arXiv preprint arXiv:2305.12433*, 2023.

Appendix A

Supplementary Diagnostic Figures

This appendix collects diagnostic figures that support the interpretation of the results in the main chapters but are not required deliverables.

Problem A

Figure A.1 backs up two claims from Chapter 1. Left: the stage-one training loss converges to the empirical variance of the 5% observation noise (dashed line), i.e. the state network fits the signal and averages the noise out; the vertical line marks the switch to stage two, where the plotted loss becomes the PDE residual. Right: the recovered flux $k_{pred} u'_{pred}$ lies on top of the exact affine flux $-f(x - 1/2)$ obtained by integrating the PDE once – the physics is satisfied to high accuracy everywhere, which confirms that the remaining error in k stems from the degeneracy at $x = 0.5$ rather than from an unsatisfied residual.

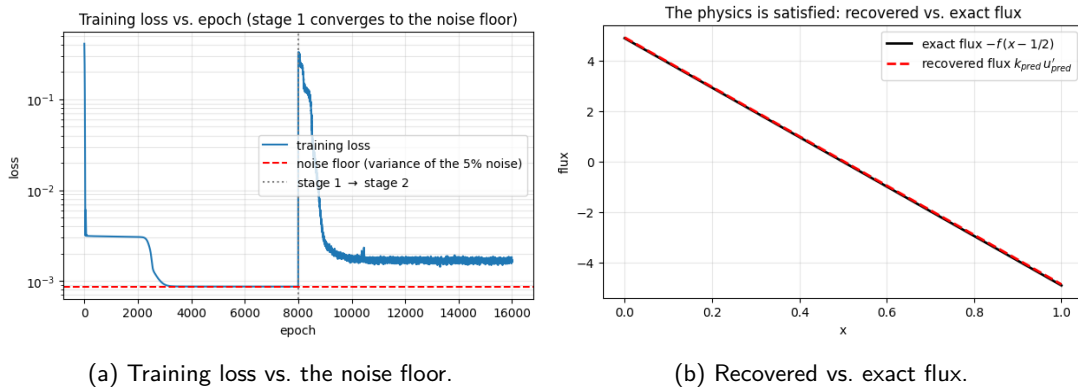


Figure A.1: Problem A diagnostics: noise-floor convergence and flux consistency.

Problem B

Figure A.2 localizes the 3.2% error of the Deep Ritz solution. Left: overlaying the phase interfaces (white contours) on the pointwise error map shows that the error is organized in thin bands along the material interfaces, as argued in Chapter 2. Right: a cross-section at $x_2 = 0.5$ makes the mechanism visible at the level of the solution profile – the reference pressure changes slope where the line crosses an inclusion, and the smooth tanh ansatz follows the profile closely while rounding off these kinks.

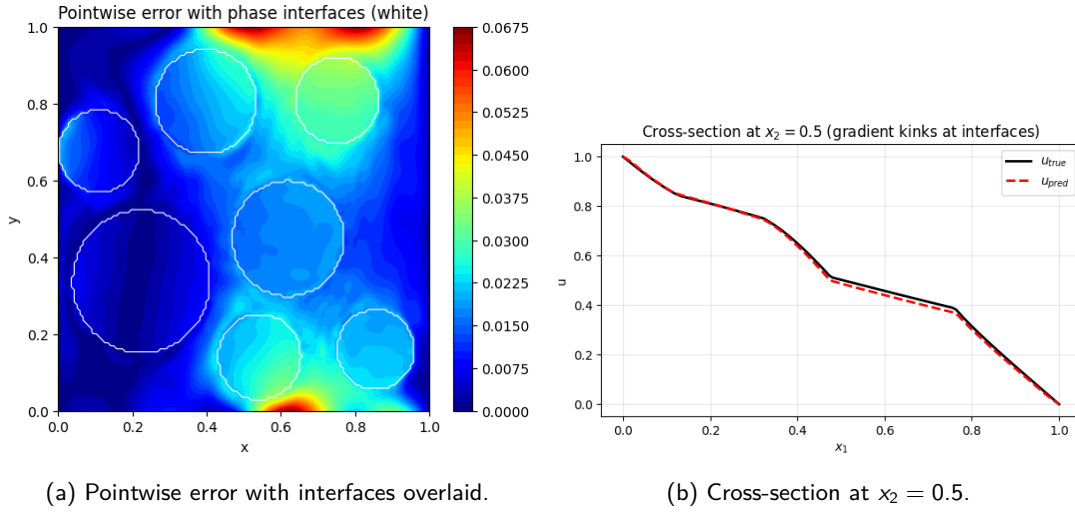


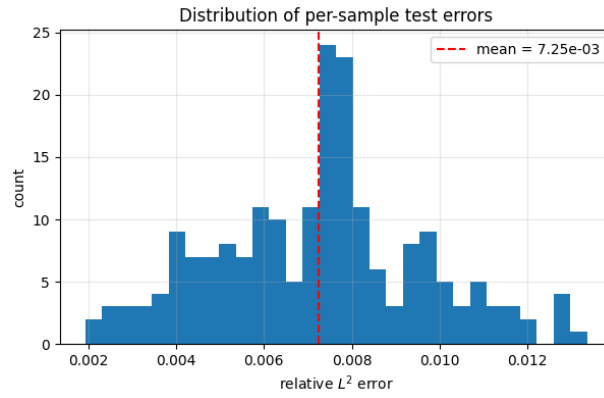
Figure A.2: Problem B diagnostics: the error is concentrated at the phase interfaces.

Problem C

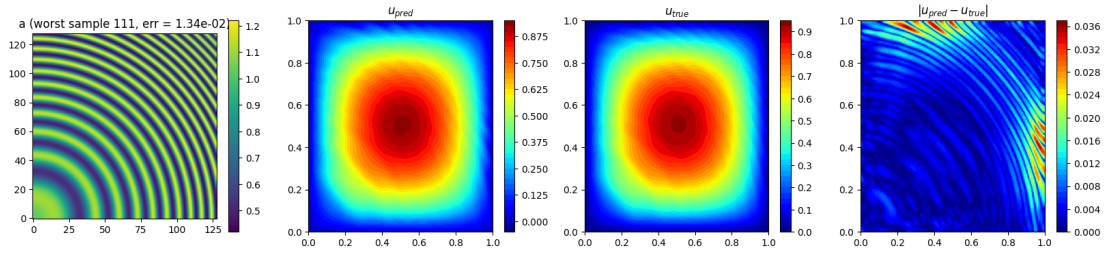
Figure A.3 characterizes the spread behind the mean test error of the FNO. The histogram of the 200 per-sample relative errors shows a compact distribution around the mean rather than a few catastrophic outliers, and the worst test sample (shown below the histogram) remains qualitatively correct – its error is elevated but of the same character as the average case, concentrated near steep conductivity gradients. Together these plots show that the reported mean is representative of the operator’s behaviour across the test distribution.

Problem D

Figure A.4 complements the space-time colour maps of Chapter 4. Left: solution profiles at $t = 0, 0.25, 0.5$, and 1.0 for the first test instance show that the PI-DeepONet tracks the steepening and subsequent viscous decay of the front, with the largest (still small) deviations at the steepest profile – consistent with the smooth basis expansion being least efficient there. Right: the per-sample error histogram is right-skewed – the bulk of the 200 test samples lies between 2% and 5% (the median is below the 4.1% mean), while a small tail of harder samples reaches 10–14% and a single outlier about 25%. The tail samples are the initial conditions that develop the steepest fronts, i.e. exactly the regime in which the smooth DeepONet basis is weakest; this skew also means the mean error reported in Chapter 4 is a conservative summary of the typical case.

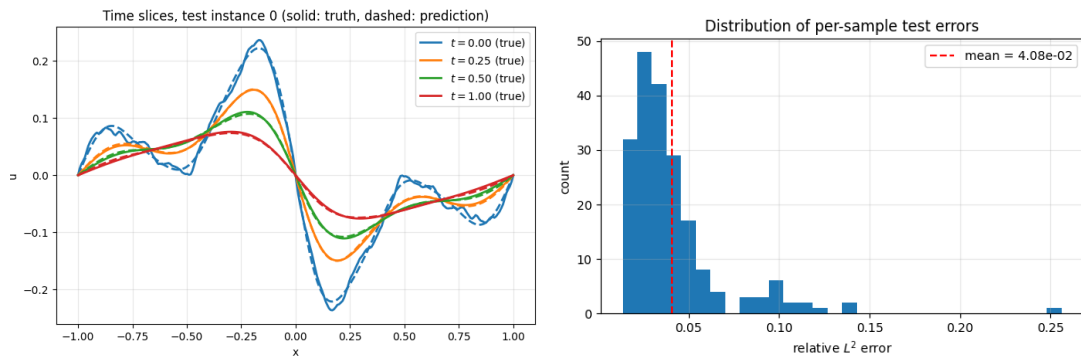


(a) Per-sample test error distribution.



(b) The worst test sample: input, prediction, reference, and pointwise error.

Figure A.3: Problem C diagnostics: error distribution and worst-case behaviour.



(a) Time slices: truth (solid) vs. prediction (dashed).

(b) Per-sample test error distribution.

Figure A.4: Problem D diagnostics: front tracking and error distribution.